

Database

Mattia Masina

CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1	Basi di Dati e Sistemi di Gestione dei Dati (DBMS)	3
1.1	Metodologia di Studio e Sistemi Informativi	3
1.2	Dati, Informazione e Codifica	5
1.3	Basi di Dati, DBMS e Confronto con i File System	6
1.4	Modelli dei Dati, Architettura e Indipendenza	10
1.5	Linguaggi per Basi di Dati e Modalità di Utilizzo	12
1.6	Attori, Ruoli, Transazioni e Valutazione dei DBMS	14
2	Il Modello Relazionale dei Dati	16
2.1	Origine, Modelli Logici e Paradigma Basato su Valori	16
2.2	Dalla Relazione Matematica alla Relazione Tabellare	17
2.3	Formalizzazione: Schemi, Istanze e Tuple	19
2.4	Strutture Dati Annidate e Appiattimento (<i>Unnesting</i>)	21
2.5	Informazione Parziale e Gestione del Valore NULL	22
2.6	Vincoli di Integrità e Loro Classificazione	24
2.7	Vincoli Intra-relazionali: Dominio e Tupla	26
2.8	Identificazione delle Tuple: Superchiavi e Chiavi	27
2.9	Chiavi, Valori NULL e la Chiave Primaria	28
2.10	Integrità Referenziale e Chiavi Esterne (<i>Foreign Key</i>)	30
2.11	Politiche di Reazione alle Violazioni e Vincoli Multipli	31

1 Basi di Dati e Sistemi di Gestione dei Dati (DBMS)

1.1 Metodologia di Studio e Sistemi Informativi

Studio Individuale e Concetti Fondamentali. Per imparare a gestire le basi di dati non basta mandare a memoria definizioni astratte. È fondamentale sviluppare una forma mentale rigorosa: capire come rappresentare la realtà in modo formale e collegare sempre la teoria a problemi pratici e casi d'uso concreti.

Attività Applicativa e Strumenti Software. La teoria deve andare di pari passo con la pratica. Esercizi, laboratori e progetti d'esame richiedono l'uso operativo di sistemi software dedicati, detti **DBMS** (*Database Management System*). Nel mondo reale e sul mercato esistono diverse categorie di DBMS:

- **Sistemi Enterprise e Commerciali:** IBM DB2, Oracle Database, Microsoft SQL Server (pensati per grandi aziende con carichi elevati).
- **Sistemi Open Source Relazionali Standard:** PostgreSQL, MySQL (molto diffusi nel web e nei progetti moderni).
- **Sistemi Desktop ed Embedded:** Microsoft Access, SQLite (leggeri, spesso integrati dentro singole applicazioni o app per smartphone).
- **Piattaforme Cloud e Analitiche:** Google BigQuery (ottimizzate per analizzare moli immense di dati).

Punto di Vista Metodologico vs Tecnologico. La materia viene affrontata da due angolazioni complementari:

- **Punto di vista metodologico:** riguarda la *progettazione*. Ci si concentra sui modelli logici e concettuali usati per descrivere le informazioni del mondo reale in modo pulito, astruendo dai dettagli fisici del computer.
- **Punto di vista tecnologico:** riguarda il *software*. Esamina come è fatto internamente un DBMS: come scrive i dati sui dischi, come gestisce la memoria, come gestisce accessi simultanei e quali linguaggi offre.

I Quattro Contenuti Fondamentali della Disciplina

- **Modelli per l'organizzazione dei dati:** insiemi di regole e strutture matematiche per descrivere entità, relazioni e vincoli del mondo reale.
- **Linguaggi per l'utilizzo dei dati:** strumenti (come SQL) per inserire, leggere, aggiornare ed eliminare i record.
- **Sistemi per la gestione dei dati:** l'architettura dei DBMS, capace di garantire sicurezza, persistenza ed efficienza.
- **Metodologie di progettazione:** i passi guidati per passare dai requisiti parlati a uno schema di database corretto, privo di errori e facile da mantenere.

Il Sistema Informativo (SI). Il **sistema informativo** è l'insieme di persone, strumenti e procedure con cui un'organizzazione (azienda, università, ospedale) raccoglie, organizza e scambia le informazioni indispensabili per svolgere le proprie attività. Anche se un'azienda non ha un reparto chiamato formalmente "Ufficio Sistema Informativo", possiede comunque un sistema informativo che ne supporta il lavoro quotidiano.

Indipendenza dall'Automazione Informatica. Un sistema informativo **non deve essere per forza computerizzato**.

Esempio: Gli archivi notarili o gli uffici anagrafe comunali dell'Ottocento funzionavano benissimo usando faldoni cartacei, registri scritti a mano e fattorini; costituivano a tutti gli effetti un sistema informativo, pur in totale assenza di computer.

Il Ciclo di Gestione dell'Informazione. Qualsiasi informazione all'interno di un'organizzazione attraversa un ciclo di quattro passaggi:

- **Raccolta e Acquisizione:** reperire i dati (es. far compilare un modulo di iscrizione a uno studente).
- **Archiviazione e Conservazione:** salvare i dati in modo duraturo per poterli ritrovare in futuro (es. registrarli in un archivio o sul disco).
- **Elaborazione e Trasformazione:** calcolare nuovi risultati a partire dai dati grezzi (es. calcolare la media dei voti o il bilancio annuale).
- **Distribuzione e Comunicazione:** trasmettere i risultati a chi ne ha bisogno (es. inviare una ricevuta via email o mostrare una schermata all'operatore).

Il Sistema Informatico (IT System). Il **sistema informatico** è la parte tecnologica e automatizzata del sistema informativo. È composto dall'hardware (server, dischi, reti), dal software (DBMS, programmi applicativi) e dagli specialisti informatici che li gestiscono.

Gerarchia dei Sistemi Aziendali: Modello a Cerchi Concentrici

Dal contesto più ampio a quello più specifico, l'organizzazione si struttura in quattro livelli concentrici:

1. **Impresa (*Enterprise*):** l'ente economico complessivo nel suo mercato (obiettivi di business, clienti, risorse).
2. **Organizzazione (*Organization*):** l'insieme coordinato di persone, uffici, compiti e processi decisionali.
3. **Sistema Informativo (*Information System*):** la parte dell'organizzazione dedicata specificamente alla gestione dei flussi informativi.
4. **Sistema Informatico (*IT System*):** l'infrastruttura tecnologica digitale (computer e software) che automatizza il sistema informativo.

Relazione di inclusione: Enterprise $\supset$ Organization $\supset$ Information System $\supset$ IT System.

Tutti gli informatici vorrebbero che il sistema informativo e quello informatico fossero perfettamente allineati. Il problema è che in certi casi non conviene o non ha senso informatizzare alcune informazioni.

1.2 Dati, Informazione e Codifica

Distinzione Concettuale e Necessità di Interpretazione. Nella vita di tutti i giorni esprimiamo concetti in molti modi: parole parlate, testo scritto, disegni, numeri o gesti. Tali informazioni possono essere salvate nella nostra memoria biologica (il cervello), su carta oppure su supporti digitali. Nei calcolatori, però, tutta questa varietà deve essere ridotta e formalizzata sotto forma di **dati**.

I due termini hanno ruoli distinti:

- **Informazione:** la conoscenza o il significato che una persona ricava da una notizia, da un fatto o da un contesto.
- **Dato:** l'elemento grezzo, preciso e privo di ambiguità (es. una stringa o un numero), memorizzato come punto di partenza per il calcolo.

Il Dato è Inutile senza Interpretazione: L'Esempio dei Segnali Finlandesi

Si considerino tre cartelli stradali finlandesi di divieto di sosta con pannelli integrativi numerici:

Dato Grezzo	Significato Reale (Informazione Compresa)
8 – 17	Divieto attivo dalle 8:00 alle 17:00 nei giorni feriali (Lunedì–Venerdì).
(8 – 13)	Divieto attivo dalle 8:00 alle 13:00 nella giornata di Sabato (indicato tra parentesi).
8 – 14	Divieto attivo dalle 8:00 alle 14:00 nei giorni festivi e Domenica (indicato in rosso).

Analisi Semantica:

I numeri da soli sono solo cifre. Senza conoscere il codice stradale finlandese (il contesto), non sappiamo a cosa si riferiscano parentesi o colori.

⇒ **Regola chiave:** Dato + Contesto / Chiave Interpretativa = Informazione.

Il Processo di Codifica dell'Informazione. Per consentire a un computer di elaborare la realtà, dobbiamo codificare le informazioni secondo regole fisse e ripetibili:

- Si prendono descrizioni del mondo reale ricche e discorsive (es. dati di un cittadino).
- Si scompone il testo in attributi precisi: *Nome, Cognome, Data di Nascita*.
- Si crea un identificatore standard sintetico e univoco: ad esempio il **Codice Fiscale**.

Perché i Dati Sono una Risorsa Strategica. All'interno di qualsiasi ente o azienda, quasi tutto cambia in fretta:

- I processi di business e le strategie commerciali si rinnovano continuamente.
- I computer, i server e i linguaggi di programmazione invecchiano e vengono rimpiazzati in pochi anni.
- Le persone cambiano ruolo o vanno in pensione.
- **I dati invece restano stabili nel tempo:** un contratto, uno storico di bonifici bancari o un registro delle nascite mantengono la propria validità e struttura invariate per decenni. I dati sono quindi il patrimonio più durevole dell'azienda.

1.3 Basi di Dati, DBMS e Confronto con i File System

Definizione di Base di Dati. L'espressione **base di dati** (*database*) viene usata con due significati:

- **Significato Generico:** qualsiasi archivio strutturato di dati usato per supportare le attività di un'organizzazione o di un individuo (anche una rubrica telefonica cartacea).
- **Significato Informatico:** una collezione organizzata di dati, salvata su memorie di massa persistenti (dischi SSD, hard disk) e gestita in modo centralizzato da un programma dedicato chiamato **DBMS**.

Il DBMS (DataBase Management System). Un **DBMS** è un sistema software concepito per gestire grandi collezioni di dati, garantendo che le operazioni siano sicure, affidabili ed efficienti.

Le Tre Proprietà Intrinseche dei Dati Gestiti da un DBMS

Un database gestito da un DBMS possiede tre proprietà distintive:

1. **Grandi Dimensioni (*Big*):** i dati non possono risiedere per intero nella memoria RAM del computer; il sistema sfrutta la memoria secondaria (dischi fisici o storage cloud).
2. **Persistenza (*Persistent*):** i dati sopravvivono alla chiusura delle applicazioni e allo spegnimento della macchina.
3. **Condivisione (*Shared*):** la stessa base di dati è usata contemporaneamente da programmi diversi e da utenti diversi, evitando di dover duplicare gli archivi per ogni ufficio.

I Quattro Requisiti di Garanzia del DBMS. Un DBMS affidabile offre quattro garanzie fondamentali:

- **Riservatezza (*Privacy* / *Sicurezza*):** ciascun utente vede e modifica solo i dati per cui ha un permesso esplicito (es. uno studente vede il proprio libretto, ma non quello degli altri).

- **Affidabilità (*Reliability*):** in caso di guasto hardware o interruzione improvvisa della corrente elettrica, i dati memorizzati non si corrompono né vanno persi.
- **Efficienza (*Efficiency*):** il sistema risponde velocemente alle richieste senza sprecare memoria o tempo di CPU, sfruttando algoritmi di indicizzazione mirati.
- **Efficacia (*Effectiveness*):** mette a disposizione strumenti intuitivi, potenti e flessibili per estrarre e lavorare sui dati utili al lavoro quotidiano.

Disponibilità dei DBMS sul Mercato. I DBMS si possono installare sui propri server (*on-premise*) oppure noleggiare come servizi gestiti nel cloud (*Database as a Service*). Alcuni dei prodotti più usati sono Oracle, Microsoft SQL Server, PostgreSQL, MySQL, IBM DB2, Google BigQuery e SQLite.

Volumetrie e Ordini di Grandezza delle Basi di Dati. Quando diciamo che i database sono “grandi”, intendiamo volumi di dati tali per cui caricare tutto il database in RAM sarebbe impossibile o insostenibile dal punto di vista dei costi.

Ordini di Grandezza nelle Basi di Dati

Tipologia di Base di Dati e Utilizzo	Dimensione Tipica
Database Operativi Quotidiani (OLTP - acquisti, pagamenti)	1 – 5 Terabyte
Database per Analisi e Report (OLAP / Data Warehouse)	30 – 50 Terabyte
Database Scientifici (meteo, fisica delle particelle)	500 – 800 Terabyte
Numero Complessivo di Record	> 100 miliardi di righe

Nota: Per archiviare queste moli di dati, l'unico vero limite considerato dal DBMS è lo spazio fisico dei dischi (SSD o magnetici) e dello storage collegato in rete.

Condivisione, Ridondanza e Inconsistenza. In un'organizzazione complessa (es. un ateneo), uffici diversi (segreteria studenti, biblioteca, mensa) lavorano spesso sulle stesse informazioni (es. l'anagrafica degli studenti). Se ogni ufficio usasse un proprio archivio separato, nascerebbero grossi problemi. La frammentazione dei dati porta a due rischi tipici:

- **Ridondanza:** lo stesso dato viene duplicato in più archivi differenti (es. l'indirizzo di casa dello studente è salvato sia nel file della segreteria sia in quello della biblioteca).
- **Inconsistenza:** le copie duplicate non coincidono tra loro.
Esempio: Lo studente cambia residenza e comunica il nuovo indirizzo solo alla segreteria; la biblioteca mantiene il vecchio indirizzo. Il sistema ora contiene due informazioni reciprocamente contraddittorie sullo stesso fatto.

Centralizzare i dati in un unico database gestito da un DBMS evita la ridondanza inutile e impedisce l'inconsistenza.

Evoluzione Storica dell'Architettura per la Gestione dei Dati. Il modo di archiviare i dati si è trasformato radicalmente nel corso dei decenni:

- **Anni '70 (Senza DBMS):** i programmi leggevano e scrivevano direttamente su file di testo o file binari tramite il sistema operativo. Non c'era separazione tra il codice e il formato del file: se il file cambiava struttura, tutti i programmi andavano riscritti.
- **Anni '80 (Primi DBMS Relazionali):** nasce il DBMS come strato intermedio. I programmi (scritti ad esempio in C o Pascal) non toccano più i file su disco, ma chiedono al DBMS le tabelle e i dati di cui hanno bisogno.
- **Anni '90 (Logica Condivisa e DBMS Attivi):** parte delle regole di controllo viene spostata dentro il database con le **stored procedure** e i **trigger** (il database reagisce in automatico a modifiche e cancellazioni).
- **Anni 2000 (Architetture Web a Tre Livelli):** separazione netta tra interfaccia e dati:

Browser (Client) $\longleftrightarrow$ Server Applicativo (Java, PHP, Python) $\longleftrightarrow$ DBMS

- **Anni 2010 (Mobile e Cloud):** decine di dispositivi diversi (app iOS, Android, web) si collegano ad API centralizzate che dialogano con basi di dati distribuite e scalabili.

Evoluzione della Logica Procedurale: Stored Procedure vs Trigger

Negli anni '90 i DBMS hanno iniziato a eseguire logica direttamente sul server:

- **Stored Procedure:** blocchi di codice salvati direttamente sul database ed eseguiti su richiesta delle applicazioni; riducono il traffico di rete ma spesso non sono facilmente portabili tra DBMS di marche diverse.
- **Trigger (DBMS Attivi):** regole automatiche che scattano da sole secondo lo schema **Evento-Condizione-Azione**.
Esempio: Quando viene inserito un nuovo ordine (Evento), se la quantità ordinata supera le scorte in magazzino (Condizione), allora blocca l'inserimento e invia una notifica (Azione).

Garanzie del DBMS: Riservatezza, Affidabilità e Transazioni. Poiché la base di dati è condivisa da molte persone, il DBMS applica regole di accesso granulari:

- *Esempio A:* Uno studente può visualizzare tutti i propri voti, ma ha il permesso di modificare soltanto la propria password e il proprio recapito telefonico.
- *Esempio B:* Il docente del corso può inserire e modificare i voti dei propri studenti, ma non può visualizzare o alterare i dati fiscali o bancari degli stessi.

Se salta la corrente o il computer va in crash mentre stiamo salvando un dato, il database non deve corrompersi né lasciare operazioni a metà. Per garantire questa affidabilità, il DBMS utilizza il meccanismo delle **transazioni**.

La Transazione: Proprietà Fondamentali (Prospettiva di Sistema)

Una **transazione** è un blocco di operazioni che il sistema tratta come un'unica unità di lavoro indivisibile, garantendo tre caratteristiche principali:

1. **Atomicità (Tutto o Niente):** o tutte le istruzioni del blocco vanno a buon fine, oppure, al minimo errore o interruzione, il sistema annulla tutto e torna allo stato iniziale.
Esempio del bonifico: Se trasferiamo 100€ dal conto A al conto B, il sistema deve togliere 100€ da A e aggiungere 100€ su B. Se la macchina va in crash subito dopo aver tolto i soldi da A, la transazione viene annullata e i 100€ ritornano su A.
2. **Coerenza e Gestione della Concorrenza:** più utenti possono operare contemporaneamente sugli stessi dati senza calpestarsi i piedi.
Esempio del posto sul treno: Se due persone tentano di prenotare l'ultimo posto disponibile nello stesso identico secondo, il DBMS assegna il posto a una sola delle due e notifica l'esaurimento all'altra, evitando il sovrapprezzo o l'*overbooking*.
3. **Permanenza (Durabilità):** una volta che l'operazione è confermata (*commit*), le modifiche sono scritte sul disco (e registrate in un file di log protetto). Anche se il computer si spegne improvvisamente un istante dopo, il dato salvato non andrà perso.

Il DBMS deve raggiungere due traguardi pratici:

- **Efficienza:** velocità di esecuzione e basso consumo di memoria, grazie a indici intelligenti (es. alberi B-Tree) e all'ottimizzatore di interrogazioni.
- **Efficacia:** fornire linguaggi semplici e potenti (come SQL) che permettano a programmatori e analisti di ottenere le risposte cercate senza dover scrivere algoritmi a basso livello.

Limiti del File System Tradizionale. Un normale sistema operativo permette di salvare dati organizzandoli in file e cartelle (**file system**).

Il limite principale dei file system è la **grana grossa**: un file va generalmente aperto, letto e riscritto per intero. Se abbiamo un file con 2 milioni di righe e vogliamo modificare il numero di telefono di una sola persona, il programma applicativo deve rileggere il file, localizzare la riga corretta in memoria e riscrivere il file sul disco. Il DBMS, al contrario, lavora a **grana fine**: individua e modifica direttamente il singolo record in frazioni di secondo.

Descrizione dei Dati e Catalogo Centralizzato. La differenza più profonda tra i due modelli sta nel modo in cui è descritta la forma dei dati:

- **Nei sistemi con File System:** ogni singolo programma deve sapere a memoria come è formattato il file (es. "i primi 20 byte contengono il cognome, i successivi 10 la data"). Se modifichiamo il formato del file aggiungendo un campo, tutti i programmi esistenti smettono di funzionare finché non vengono riscritti.
- **Nei DBMS:** la struttura dei dati è definita in un solo posto centrale chiamato **catalogo** (o **dizionario dei dati**). I programmi non devono conoscere i dettagli fisici di memorizzazione: chiedono semplicemente le informazioni per nome al DBMS.

1.4 Modelli dei Dati, Architettura e Indipendenza

Il Modello dei Dati. Un **modello dei dati** è una collezione di concetti usati per organizzare e descrivere i dati.

Ogni modello offre specifici **costruttori di tipo**, simili a quelli usati nella programmazione (come array o struct). Nel **modello relazionale**, il costruttore principale è la **relazione** (comunemente vista come una **tabella**), composta da righe omogenee (tuple).

Schema e Istanza. In qualunque base di dati si distingue nettamente tra forma e contenuto:

- **Schema (Aspetto Intensionale):** l'intestazione, la struttura, i tipi di dato e i vincoli delle tabelle. È fisso e quasi mai soggetto a modifiche nel tempo.
- **Istanza (Aspetto Estensionale):** i dati concreti memorizzati nelle righe in un determinato momento. È molto dinamica, poiché righe vengono inserite, aggiornate o cancellate di continuo.

Esempio: Schema e Istanza nella Tabella Orario Lezioni

Si consideri una semplice tabella che organizza l'orario delle lezioni:

Schema della Relazione (Intestazione fissa, definita una volta sola)			
Corso (<i>Course</i>)	Docente (<i>Lecturer</i>)	Aula (<i>Room</i>)	Ora (<i>Time</i>)
Istanza della Relazione (Le righe di dati salvate in questo momento)			
Analisi I	Luigi Neri	N1	8:00
Basi di Dati	Pier Rossi	N2	9:45
Chimica	Nicola Mori	N1	9:45
Fisica I	Mario Bruni	N1	11:45
Fisica II	Mario Bruni	N3	9:45
Sistemi Informatici	Pier Rossi	N3	8:00

Le intestazioni definiscono **cosa** può contenere la tabella (lo *schema*); le sei righe di valori rappresentano il contenuto effettivo attuale (l'*istanza*).

Modelli Logici vs Modelli Concettuali. Esistono due grandi famiglie di modelli usate in momenti diversi della progettazione:

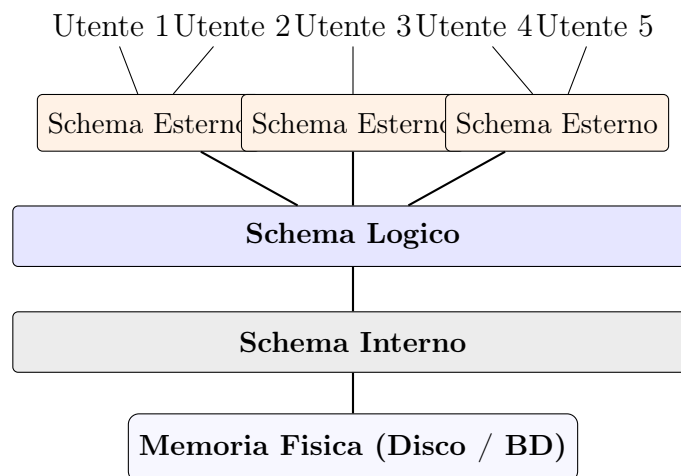
- **Modelli Logici:** sono i modelli compresi e usati dai DBMS per organizzare i dati su disco. Sono indipendenti dai dettagli fisici del disco, ma già vicini all'implementazione software.
Esempi: il modello **relazionale** (a tabelle), il modello gerarchico, a oggetti o documentale/-JSON (NoSQL).
- **Modelli Concettuali:** sono formalismi grafici e intuitivi usati per dialogare con i commitenti e descrivere la realtà senza pensare a quale software verrà usato in futuro.
Il più celebre: il **Modello Entità-Relazione (E-R)**.

Architettura dei DBMS: Livelli e Astrazione. In prima approssimazione, un sistema di gestione dati separa due aspetti (Architettura Semplificata a Due Livelli):

- **Schema Logico:** la vista complessiva della struttura (quali tabelle esistono, quali colonne contengono, quali relazioni ci sono tra loro).
- **Schema Interno (o Fisico):** come quei record sono disposti sui blocchi del disco, quali puntatori e quali file fisici vengono usati.

Architettura Standard ANSI/SPARC a Tre Livelli. Articola la visione dei dati su tre livelli:

1. **Livello Esterno (Schemi Esterni / Viste):** schermate e finestre ritagliate su misura per utenti specifici. Ogni gruppo di utenti vede solo i dati di proprio interesse.
2. **Livello Logico (Schema Logico):** la descrizione completa, unica e priva di ridondanze dell'intera base di dati.
3. **Livello Interno (Schema Interno):** l'organizzazione fisica su disco, i file di memoria e le strutture di indicizzazione.



Il Meccanismo delle Viste. Una **vista** è una tabella virtuale: non memorizza dati duplicati su disco, ma calcola le righe al momento eseguendo una query sulle tabelle reali sottostanti.

Esempio di Costruzione di una Vista: Accoppiamento Corsi-Aule

Consideriamo due tabelle di base memorizzate nel database:

Tabella Lezioni			Tabella Aule		
Corso	Docente	Aula	Nome	Edificio	Piano
Basi di Dati	Rossi	DS3	DS1	OMI	Terra
Sistemi	Neri	N3	N3	OMI	Terra
Reti	Bruni	N3	G	Pincherle	Primo
Controlli	Bruni	G			

Se agli studenti interessa solo sapere dove andare a lezione (senza vedere il nome del docente o le aule vuote), il DBMS può creare al volo la vista **AuleLezioni**, collegando le due tabelle tramite l'aula:

Corso	Aula	Edificio	Piano
Sistemi	N3	OMI	Terra
Reti	N3	OMI	Terra
Controlli	G	Pincherle	Primo

In questo modo l'utente vede una tabella pulita e pronta all'uso, senza che i dati siano stati duplicati sul disco.

Le Due Forme di Indipendenza dei Dati. Separare i tre livelli garantisce il principio di indipendenza dei dati:

- **Indipendenza Fisica:** possiamo cambiare il modo in cui i file sono scritti su disco, spostare il database su un SSD più veloce o aggiungere un indice per velocizzare le ricerche, **senza dover cambiare una sola riga** dei programmi applicativi o delle query SQL.
- **Indipendenza Logica:** possiamo aggiungere nuove tabelle o nuove colonne al database **senza rompere** i programmi preesistenti, che continuano a funzionare interagendo con le loro viste abituali.

1.5 Linguaggi per Basi di Dati e Modalità di Utilizzo

Classificazione Funzionale: DDL vs DML. Nei database si usano due categorie distinte di comandi:

- **DDL (Data Definition Language):** comandi per creare o modificare la struttura delle tabelle (lo schema).
Esempi: CREATE TABLE, ALTER TABLE, DROP TABLE.
- **DML (Data Manipulation Language):** comandi per inserire, cercare, modificare ed eliminare i record effettivi (l'istanza).
Esempi: SELECT, INSERT, UPDATE, DELETE.

```

1 CREATE TABLE orario (
2     corso    CHAR(20),
3     docente  CHAR(20),
4     aula     CHAR(4),
5     ora      CHAR(5)
6 );

```

Listing 1: Esempio DDL: creazione dello schema di una tabella

I Quattro Paradigmi di Interazione con la Base di Dati. Un utente o un programma può dialogare con il DBMS in quattro modi:

1. Linguaggio Interattivo Testuale (SQL a riga di comando). L'utente scrive direttamente comandi SQL in una console per ottenere subito i risultati a video:

```

1 SELECT Corso, Aula, Piano
2 FROM Aule, Lezioni
3 WHERE Nome = Aula
4     AND Piano = 'Terra';

```

Listing 2: Query SQL interattiva

2. SQL Immerso in un Linguaggio Ospite (*Embedded SQL*). I comandi SQL vengono inseriti all'interno di un linguaggio tradizionale (es. C o Pascal). Poiché SQL restituisce insiemi di righe mentre le variabili dei linguaggi procedurali leggono un valore alla volta, si usa un **cursore** per scorrere i risultati riga per riga:

```

1 write('Nome della citta? '); readln(citta);
2 EXEC SQL DECLARE P CURSOR FOR
3     SELECT NAME, INCOME
4     FROM PEOPLE
5     WHERE CITY = :citta;
6 EXEC SQL OPEN P;
7 EXEC SQL FETCH P INTO :name, :income;
8 while SQLCODE = 0 do begin
9     write('Nome persona: ', name, ' Aumento? ');
10    readln(aumento);
11    EXEC SQL UPDATE PEOPLE
12        SET INCOME = INCOME + :aumento
13        WHERE CURRENT OF P;
14    EXEC SQL FETCH P INTO :name, :income;
15 end;
16 EXEC SQL CLOSE CURSOR P;

```

Listing 3: Esempio di Embedded SQL in Pascal con cursore

3. Linguaggi Procedurali Integrati nel DBMS (es. Oracle PL/SQL). Si tratta di linguaggi che uniscono istruzioni SQL a costrutti classici di programmazione (IF, cicli WHILE, gestione degli errori):

```

1 DECLARE
2     Stip NUMBER;
3 BEGIN
4     SELECT SALARY INTO Stip FROM EMPLOYEE
5     WHERE NUMBER = '575488' FOR UPDATE OF SALARY;
6
7     IF Stip > 30 THEN
8         UPDATE EMPLOYEE SET SALARY = SALARY * 1.1
9         WHERE NUMBER = '575488';
10    ELSE
11        UPDATE EMPLOYEE SET SALARY = SALARY * 1.15
12        WHERE NUMBER = '575488';
13    END IF;
14
15    COMMIT;
16 EXCEPTION
17    WHEN NO_DATA_FOUND THEN
18        INSERT INTO ERRORS

```

```
19      VALUES ( 'MATRICOLA NON ESISTENTE' , SYSDATE ) ;  
20 END ;
```

Listing 4: Esempio di blocco procedurale PL/SQL

4. Interfacce Grafiche (GUI e Maschere a Schermo). Gli utenti non tecnici (es. operatori di sportello, cassieri) non scrivono comandi SQL: usano maschere grafiche con campi da compilare, pulsanti e menu a tendina (es. Microsoft Access o pagine web gestionali). Dietro le quinte, il software genera ed esegue i comandi SQL corrispondenti.

1.6 Attori, Ruoli, Transazioni e Valutazione dei DBMS

I Quattro Gruppi di Ruoli Operativi. Nel mondo delle basi di dati lavorano quattro figure principali:

1. **Costruttori del DBMS:** gli ingegneri informatici che scrivono il motore del DBMS (progettano gli ottimizzatori di query, la gestione della memoria su disco e i compilatori interni).
2. **Progettisti del Database e DBA:** chi decide come strutturare le tabelle per uno specifico problema e ne amministra il funzionamento quotidiano.
3. **Sviluppatori Applicativi:** i programmatori che scrivono i programmi e le interfacce utente per interagire con i dati.
4. **Utenti:**
 - *Utenti Finali (Operativi):* usano interfacce guidate e maschere predefinite per compiti di routine (es. l'impiegato postale che registra un pacco).
 - *Utenti Casuali (Casual Users):* analisti o manager che interrogano il database formulando query SQL personalizzate per prendere decisioni strategiche.

L'Amministratore della Base di Dati (DBA). Il DBA (*Database Administrator*) è il custode e responsabile dell'efficienza e della salute del database. I suoi compiti principali sono:

- Monitorare le prestazioni del sistema e configurare gli indici per non rallentare le ricerche.
- Pianificare i salvataggi periodici (**backup**) e le procedure di ripristino dei dati in caso di guasti o disastri (*disaster recovery*).
- Gestire gli account, le password e le autorizzazioni di sicurezza di tutti gli utenti.
- Spesso collabora anche alla progettazione iniziale e all'evoluzione delle tabelle.

I Due Punti di Vista sulla Transazione. La parola “transazione” ha due sfumature:

- **Per l'utente finale:** è un'azione applicativa compiuta con uno scopo chiaro (es. fare un bonifico, comprare un biglietto aereo, iscriversi a un esame).
- **Per il DBMS:** è una serie indivisibile di comandi che devono essere eseguiti con successo al 100% oppure annullati del tutto, salvaguardando la coerenza dell'archivio anche in caso di errori o accessi concorrenti.

I Vantaggi (Pros) nell'Adozione di un DBMS. Scegliere un DBMS porta chiari vantaggi:

- **Dati Condivisi:** un'unica fonte di verità centralizzata per tutti i reparti dell'azienda.
- **Niente Duplicazioni Inutili:** si eliminano i rischi di avere dati disallineati e contraddittori.
- **Servizi Integrati:** il DBMS gestisce da solo sicurezza, controllo della concorrenza, transazioni e salvataggi.
- **Indipendenza dei Dati:** se si modifica l'hardware o la disposizione fisica dei file, non serve riscrivere i software applicativi.

Gli Svantaggi e i Costi (Cons). Un DBMS non è sempre la soluzione ideale in qualunque scenario, a causa di due fattori:

- **Costi Economici e Complessità:** le licenze enterprise sono onerose, richiedono server potenti e personale altamente qualificato (i DBA) per l'amministrazione.
- **Struttura Impegnativa (*Overhead*):** le funzioni di controllo transazionale, sicurezza e logging sono sempre attive e consumano risorse. Per compiti minuscoli o file di log elementari, un semplice file di testo gestito dal sistema operativo può risultare più veloce ed economico rispetto a un intero DBMS.

2 Il Modello Relazionale dei Dati

2.1 Origine, Modelli Logici e Paradigma Basato su Valori

Evoluzione dei Modelli Logici Tradizionali. Prima dell'avvento del modello relazionale, la gestione dei dati nelle applicazioni aziendali si basava su due grandi modelli logici:

- **Modello Gerarchico:** organizza i record secondo strutture ad albero piramidali; ogni record figlio possiede un unico record genitore, replicando fedelmente le gerarchie rigide ma rendendo complessa la gestione di relazioni molti-a-molti.
- **Modello Reticolare:** generalizza il modello gerarchico consentendo strutture a grafo; un record può avere più record proprietari (*owner*) e record membri (*member*).

Entrambi questi modelli presentano un limite strutturale insidioso: le associazioni tra i dati sono espresse mediante **riferimenti espliciti** (puntatori di memoria fisici o logici gestiti a basso livello). Chi programma deve conoscere nel dettaglio la topologia dei collegamenti per navigare l'archivio record per record.

Modelli Alternativi e Successivi. Negli anni successivi sono emerse ulteriori proposte:

- **Modello a Oggetti (*Object-Oriented*):** estende ai database i concetti della programmazione a oggetti (incapsulamento, metodi, ereditarietà, identità tramite OID). Sebbene potente, ha trovato diffusione solo in nicchie specifiche e complesse (es. CAD/CAM, sistemi geografici).
- **Modello Documentale e Semistutturato (XML / JSON):** nato per gestire dati privi di uno schema rigido a priori o fortemente gerarchici, oggi ampiamente adottato nei sistemi NoSQL ed è da considerarsi complementare al modello relazionale.

La Rivoluzione di Edgar F. Codd (1970). Il modello relazionale fu proposto formalmente da **Edgar F. Codd** nel celebre articolo del 1970 con un obiettivo primario: raggiungere la piena **indipendenza logica e fisica dei dati**. Nei modelli basati su puntatori, ogni modifica alla struttura interna dei file su disco obbligava a riscrivere il software applicativo. Codd propose di rappresentare l'intera base di dati mediante una struttura concettuale pulita e matematica: la **relazione**.

L'adozione commerciale effettiva arrivò circa un decennio dopo, nel **1981**, poiché garantire l'indipendenza dei dati senza sacrificare le prestazioni computazionali e l'affidabilità su memorie di massa richiese anni di ricerca algoritmica (in particolare per lo sviluppo di optimizer di query e indici su albero).

Puntatori Espliciti vs Paradigma Basato su Valori (*Value-Based*)

La distinzione cruciale tra i sistemi pre-relazionali e il modello relazionale risiede nel meccanismo con cui si stabiliscono i legami tra le entità:

- **Sistemi basati su puntatori (Gerarchico/Reticolare):** i collegamenti tra record diversi sono rappresentati da indirizzi o identificatori fisici gestiti dal motore di sistema. Il legame è intrinsecamente **direzionale** (un puntatore va da un record *A* a un record

B , ma per risalire da B ad A occorrono strutture aggiuntive).

- **Modello relazionale (Basato su Valori):** le correlazioni tra i dati memorizzati in relazioni diverse sono espresse **esclusivamente attraverso i valori** contenuti nelle tuple stesse.

Esempio concreto: se una tabella **Esami** deve indicare quale studente ha sostenuto la prova, essa non conterrà l'indirizzo su disco del record dello studente, bensì il valore della sua matricola (es. 3456). Tale valore coincide con la chiave memorizzata nella tabella **Studenti**.

I Vantaggi Chiave della Struttura Basata su Valori. L'adozione del paradigma *value-based* porta benefici sostanziali:

- **Indipendenza dalla struttura fisica:** i puntatori fisici cambiano quando i dati vengono riallocati o deframmentati su disco; i valori informativi rimangono inalterati indipendentemente dal supporto hardware.
- **Pertinenza applicativa:** nel database si salvano esclusivamente i dati che hanno un chiaro significato informativo per il dominio applicativo, scartando metadati di navigazione fittizi.
- **Visione uniforme per utente e sviluppatore:** chi interroga la base di dati osserva le medesime strutture tabellari su cui ragiona il programmatore, senza dettagli nascosti.
- **Interoperabilità e portabilità:** dati basati su valori possono essere esportati, serializzati e scambiati con facilità tra piattaforme e DBMS eterogenei.
- **Simmetria e navigazione bidirezionale:** mentre un puntatore impone una traiettoria prefissata, l'uguaglianza sui valori consente di navigare la correlazione con pari facilità in entrambi i versi (trovare tutti gli esami sostenuti dallo studente X oppure risalire all'anagrafica dello studente che ha superato l'esame Y).

2.2 Dalla Relazione Matematica alla Relazione Tabellare

Chiarimento Terminologico Fondamentale. L'uso del termine "relazione" genera spesso equivoci tra mondi contigui:

- **Relazione logico-matematica (Teoria degli Insiemi):** sottoinsieme del prodotto cartesiano tra insiemi di elementi.
- **Relazione nel Modello Relazionale:** costruito logico rappresentabile come una tabella bidimensionale con intestazioni dotate di significato.
- **Relationship (Associazione o Correlazione):** concetto del **Modello Entità-Relazione (E-R)**, che descrive una classe di fatti o un legame concettuale tra due o più entità del mondo reale.

La Relazione Matematica Insiemistica. Siano $D_1, D_2, \dots, D_n$ insiemi detti **domini** (non necessariamente distinti tra loro).

- Il **prodotto cartesiano** $D_1 \times D_2 \times \dots \times D_n$ è l'insieme di tutte le n -uple ordinate $(d_1, d_2, \dots, d_n)$ tali per cui:

$$d_1 \in D_1, \quad d_2 \in D_2, \quad \dots, \quad d_n \in D_n$$

- Una **relazione matematica** r sui domini $D_1, \dots, D_n$ è un qualsiasi sottoinsieme del loro prodotto cartesiano:

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

I domini $D_1, \dots, D_n$ prendono il nome di **domini della relazione**, mentre il numero n rappresenta il **grado** della relazione.

Proprietà della Relazione Insiemistica: Notazione Posizionale

In base alla rigorosa definizione della teoria degli insiemi:

1. **Una relazione è un insieme:** ne discende che tra le ennuple non esiste alcun ordinamento intrinseco e che non possono esistere due ennuple identiche (le tuple sono tutte distinte).
2. **Ogni ennupla è una sequenza ordinata:** la posizione dell' i -esimo elemento determina in modo rigido da quale dominio esso provenga. L'accesso ai valori si basa sulla loro **posizione ordinale** ($1^\circ, 2^\circ, \dots, n^\circ$).

Il Limite della Posizionalità: L'Esempio del Calendario Partite. Si consideri una relazione $Partite \subseteq \text{String} \times \text{String} \times \text{Int} \times \text{Int}$ che descrive i risultati di un girone calcistico:

1° Componente	2° Componente	3° Componente	4° Componente
Barcellona	Bayern	3	1
Real Madrid	Bayern	2	0
Barcellona	Juventus	0	2
Juventus	Real Madrid	0	1

In questo schema insiemistico, il dominio String appare due volte e il dominio Int compare due volte. Il loro ruolo semantico è affidato unicamente alla posizione:

- Posizione 1: squadra ospitante (*Home*).
- Posizione 2: squadra ospitata (*Away*).
- Posizione 3: reti realizzate dalla squadra di casa (*GoalsH*).
- Posizione 4: reti realizzate dalla squadra ospite (*GoalsA*).

Se scambiassimo di posto le prime due colonne senza mutare i dati, il significato del record risulterebbe stravolto.

Dalla Struttura Posizionale alla Struttura Non Posizionale. Nei database reali, affidare il significato alla posizione è rischioso e scomodo. Il modello relazionale supera questo limite introducendo gli **attributi**: a ciascuna colonna viene assegnato un nome simbolico univoco che ne definisce formalmente il ruolo.

L'Attributo e l'Indipendenza dall'Ordine delle Colonne

Un **attributo** è un nome unico associato a un dominio per esplicitarne il significato applicativo all'interno della relazione.

Con l'introduzione degli attributi, la tabella precedente viene modellata come:

SquadraCasa	SquadraOspite	GolCasa	GolOspite
Barcellona	Bayern	3	1
Bayern	Real Madrid	2	0
Barcellona	Juventus	0	2
Juventus	Real Madrid	0	1

Poiché ciascun dato è identificato dal nome dell'attributo e non dal suo indice numerico, **l'ordine orizzontale delle colonne è del tutto irrilevante:**

SquadraOspite	SquadraCasa	GolOspite	GolCasa
Bayern	Barcellona	1	3
Real Madrid	Bayern	0	2
Juventus	Barcellona	2	0
Real Madrid	Juventus	1	0

Entrambe le tabelle rappresentano la medesima informazione logica: la struttura dei dati è pienamente **non posizionale**.

Proprietà di una Tabella come Relazione. Una tabella su video o su carta rappresenta una valida relazione relazionale se e solo se rispetta le seguenti condizioni:

1. Tutte le righe sono distinte tra loro (nessuna tupla duplicata).
2. L'ordine delle righe è irrilevante.
3. Tutte le intestazioni di colonna (attributi) sono distinte all'interno della stessa tabella.
4. L'ordine delle colonne è irrilevante.
5. I valori all'interno di ciascuna colonna sono omogenei rispetto al dominio dell'attributo.

2.3 Formalizzazione: Schemi, Istanze e Tuple

Schema di Relazione. Uno **schema di relazione** $R(X)$ è costituito da:

- Un nome simbolico R (es. STUDENTI).
- Un insieme di nomi di attributi $X = \{A_1, A_2, \dots, A_n\}$.
- A ciascun attributo A_i è associato un insieme di valori ammissibili, denominato dominio $\text{dom}(A_i)$.

Viene compattamente indicato con $R(A_1, A_2, \dots, A_n)$.

Schema di Base di Dati. Uno **schema di base di dati relazionale** $\mathcal{R}$ è una collezione finita di schemi di relazione con nomi distinti:

$$\mathcal{R} = \{R_1(X_1), R_2(X_2), \dots, R_k(X_k)\}$$

Definizione Formale di Tupla. Data una relazione con schema $R(X)$, una **tupla** t su X non è una semplice sequenza ordinata, bensì una **funzione matematica** che mappa ogni attributo $A \in X$ in un valore del suo rispettivo dominio:

$$t : X \longrightarrow \bigcup_{A \in X} \text{dom}(A) \quad \text{tale che} \quad t[A] \in \text{dom}(A) \quad \forall A \in X$$

La notazione $t[A]$ (oppure $t.A$) indica il valore assunto dalla tupla t in corrispondenza dell'attributo A .

Esempio Completo: Schemi e Istanze in un Contesto Didattico

Si consideri lo schema di base di dati universitario:

$$\mathcal{R} = \{\text{STUDENTI}(\text{Matricola}, \text{Cognome}, \text{Nome}, \text{DataNascita}), \\ \text{CORSI}(\text{Codice}, \text{NomeCorso}, \text{Docente}), \\ \text{ESAMI}(\text{Candidato}, \text{Voto}, \text{Corso})\}$$

Una possibile **istanza di relazione** per la tabella **STUDENTI** è l'insieme di tuple:

Tupla	Matricola	Cognome	Nome	DataNascita
t_1	6554	Rossi	Mario	1978/12/05
t_2	8765	Neri	Paolo	1976/11/03
t_3	9283	Verdi	Luisa	1979/11/12
t_4	3456	Rossi	Maria	1978/02/01

Se consideriamo la prima tupla t_1 , possiamo estrarre i singoli componenti mediante la notazione funzionale:

$$t_1[\text{Matricola}] = 6554, \quad t_1[\text{Cognome}] = \text{'Rossi'}, \quad t_1[\text{DataNascita}] = \text{'1978/12/05'}$$

Istanza di Relazione e di Base di Dati.

- Un'**istanza di relazione** r su uno schema $R(X)$ è un insieme finito di tuple definite su X .
- Un'**istanza di base di dati** su uno schema $\mathcal{R} = \{R_1(X_1), \dots, R_k(X_k)\}$ è l'insieme delle rispettive istanze di relazione:

$$r = \{r_1, r_2, \dots, r_k\}$$

dove ogni r_i è un'istanza conforme allo schema $R_i(X_i)$.

Relazioni con un Solo Attributo (Unarie). Una relazione può essere definita su un unico attributo ($n = 1$). Tali relazioni sono molto utili per rappresentare sottoinsiemi, stati o proprietà del dominio senza dover introdurre colonne booleane ridondanti.

STUDENTI_LAVORATORI
Matricola
6554
3456

La presenza del valore indica l'appartenenza dello studente alla categoria dei lavoratori.

2.4 Strutture Dati Annidate e Appiattimento (*Unnesting*)

La Complessità dei Dati del Mondo Reale. Nei documenti cartacei e nelle ricevute contabili i dati si presentano spesso in forma gerarchica e ricorsiva. Si pensi a uno scontrino rilasciato da un ristorante:

- Informazioni di testata generali: nome del locale, indirizzo, data, numero scontrino fiscale e conto totale.
- Corpo dello scontrino: una lista variabile di consumazioni (ciascuna con quantità, descrizione della portata e prezzo unitario o complessivo).

Il Vincolo della Prima Forma Normale (1NF). Nel modello relazionale canonico vige il vincolo di **atomicità dei domini**: i valori contenuti all'incrocio tra una riga e una colonna devono essere indivisibili. Non è consentito inserire come valore di un attributo una lista, un vettore o una sotto-tabella (strutture non in *First Normal Form* o $\neg 1NF$).

Il Processo di Decomposizione e Appiattimento (*Unnesting*)

Per rappresentare fedelmente uno scontrino nel modello relazionale, la struttura annidata deve essere scomposta (*unnested*) in due relazioni distinte ma correlate per valore:

Relazione RICEVUTE		
Numero	Data	Totale
1235	12/10/2002	39,20
1240	13/10/2002	39,00

Relazione PORTATE (Dettaglio Scontrino)

Ricevuta	Riga	Descrizione	Prezzo
1235	1	3 Coperti	3,00
1235	2	2 Antipasti	6,20
1235	3	3 Primi	12,00
1235	4	2 Bistecche	18,00
1240	1	2 Coperti	2,00
1240	2	2 Antipasti	7,00

Analisi delle Scelte Progettuali:

- Il campo **Ricevuta** nella tabella **PORTATE** funge da connettore logico verso la tabella **RICEVUTE**.
- È stato introdotto l'attributo esplicito **Riga** per preservare l'ordinamento originario con cui i piatti sono stati battuti alla cassa. Poiché il modello relazionale non assegna alcun significato all'ordine di memorizzazione delle righe, l'unico modo per tracciare una sequenza è renderla un dato informativo esplicito.
- L'introduzione della riga d'ordine consente inoltre di ammettere portate duplicate consumate in momenti diversi dello stesso pasto (es. due ordinazioni successive della stessa bevanda).

2.5 Informazione Parziale e Gestione del Valore NULL

La Rigidità dello Schema e il Problema dei Valori Mancanti. Il modello relazionale impone una struttura stretta: ogni tupla conforme a uno schema $R(A_1, \dots, A_n)$ deve possedere un valore per ciascuno degli attributi previsti. Nella realtà dei fatti, tuttavia, le informazioni raccolte possono essere incomplete o temporaneamente non disponibili.

Si consideri l'anagrafica di quattro personaggi storici memorizzata in uno schema composto da **Nome**, **SecondoNome** (*Middle Name*) e **Cognome**:

Nome	SecondoNome	Cognome
Franklin	Delano	Roosevelt
Winston	?	Churchill
Charles	?	De Gaulle
Josip	?	Stalin

Franklin D. Roosevelt possiede effettivamente un secondo nome, mentre gli altri statisti ne sono sprovvisti o non è registrato. Come gestire la casella vuota?

Perché Non Usare Valori Sentinella del Dominio. Un approccio ingenuo consiste nell'adottare valori speciali appartenenti al dominio (es. stringa vuota "", numeri convenzionali come 0, -1 o 9999). Questa tecnica presenta gravi difetti:

- **Mancanza di valori disponibili:** in molti domini continui o aperti non esistono valori non validi o inutilizzati a priori.
- **Collisioni future:** valori scelti come sentinella potrebbero acquisire un significato reale nel tempo (es. uno zero per la temperatura o per un saldo bancario non significa assenza di dato).
- **Onere applicativo ed errori di calcolo:** i programmatori devono ricordare a memoria le sentinelle e gestirle manualmente nel codice con condizioni **IF**. Se la sentinella non viene filtrata, altera funzioni aggregate come medie e conteggi.

La Soluzione Relazionale: Il Marcatore Speciale NULL

Per superare i limiti delle sentinelle, il modello relazionale introduce un costrutto universale dedicato: il valore **NULL**.

- NULL denota in modo formale l'**assenza di un valore informativo valido**.
- NULL **non appartiene ad alcun dominio di tipo**: si estende la definizione formale di tupla ammettendo che:

$$t[A] \in \text{dom}(A) \cup \{\text{NULL}\}$$

- Nei database moderni, l'ammissibilità dei valori NULL può essere inibita specificando nello schema vincoli dedicati (es. la clausola **NOT NULL** del linguaggio SQL).

Le Tre Sfumature Semantiche del Valore Mancante. Dal punto di vista concettuale, l'assenza di un dato può manifestarsi sotto tre forme profondamente differenti:

1. **Valore Sconosciuto (*Unknown*)**: il valore esiste realmente nel mondo reale, ma al momento della registrazione non è noto al sistema.
Esempio: la data di nascita o l'indirizzo di residenza di una persona che non ha ancora compilato il modulo anagrafico.
2. **Valore Inesistente o Non Applicabile (*Inexistent / Not Applicable*)**: l'attributo non ha alcuna ragione logica di esistere per quella specifica entità.
Esempio: il numero di patente di guida per un individuo che non ha mai conseguito la licenza, o il cognome da nubile per un soggetto di sesso maschile.
3. **Valore Non Informativo o Riservato (*No Information*)**: non è possibile determinare se il dato esista o meno, oppure la legislazione sulla privacy ne preclude l'archiviazione.

Trattamento nei DBMS e il Rischio di Proliferazione (*Too Many NULLs*). Sebbene la teoria riconosca la triplice sfumatura di significato, i DBMS commerciali non distinguono internamente tra valore sconosciuto e valore inesistente: entrambi sono registrati indistintamente tramite il bit di marcatura NULL.

L'abuso dei valori NULL introduce severe complicazioni:

- Richiede l'adozione di una logica a tre valori di verità (*True, False, Unknown*) nell'elaborazione dei predicati delle query SQL.
- Rende ambigue le operazioni di correlazione e join tra tabelle diverse.
- Se una tupla accumula troppi valori NULL (come illustrato nella tabella sottostante), perde la sua identità informativa e cessa di essere utile all'applicazione.

Tabella STUDENTI con Eccessiva Presenza di Valori NULL			
Matricola	Cognome	Nome	DataNascita
6554	Rossi	Mario	1978/12/05
8765	Neri	Paolo	1976/11/03
9283	Verdi	Luisa	1979/11/12
NULL	Rossi	Maria	1978/02/01

Come evidenziato nell'ultima riga, se il valore NULL si presenta su un attributo deputato a identificare la riga (la matricola), il record non può più essere collegato in modo univoco agli esami sostenuti o ad altre tabelle del database. Nasce così la necessità di regole strutturali rigide: i **vincoli di integrità**.

2.6 Vincoli di Integrità e Loro Classificazione

Limiti della Correttezza Sintattica. Non tutte le istanze di basi di dati sintatticamente conformi a uno schema descrivono una realtà plausibile o corretta per l'applicazione. Si consideri la seguente istanza di prova:

Relazione ESAMI				Relazione STUDENTI
Matricola	Voto	Lode	Corso	
276545	32		01	
276545	30	Lode	02	
787643	27	Lode	03	
739430	24		04	

Matricola	Cognome	Nome
276545	Rossi	Mario
787643	Neri	Piero
787643	Bianchi	Luca

Dal punto di vista dei tipi di dato l'istanza è impeccabile (i voti sono interi, i nomi sono stringhe), eppure contiene gravi anomalie semantiche:

- Un voto d'esame pari a 32, valore inammissibile nella scala di valutazione universitaria ($18 \leq \text{voto} \leq 30$).
- L'assegnazione della lode a una prova superata con 27, combinazione priva di senso regolamentare.
- La presenza di due studenti differenti registrati con la medesima matricola (787643).

Definizione Formale di Vincolo di Integrità

Un **vincolo di integrità** è un predicato logico (una funzione booleana) associato a uno schema di base di dati che associa a ogni possibile istanza di relazione o di base di dati il

valore di verità **Vero** (*True*) o **Falso** (*False*):

$$\mathcal{C} : \text{Istanze}(R) \longrightarrow \{\text{True}, \text{False}\}$$

- Un'istanza di base di dati si definisce **valida** (o **legale**) se soddisfa tutti i vincoli di integrità definiti sullo schema ($\mathcal{C}(r) = \text{True}$).
- Un'istanza è **non valida** se viola anche un solo vincolo ($\mathcal{C}(r) = \text{False}$).

Motivazioni per l'Introduzione dei Vincoli. La definizione esplicita dei vincoli è essenziale per molteplici scopi:

- **Accuratezza Semantica:** consente di descrivere in modo rigoroso e formale le regole del mondo reale modellato nel database.
- **Qualità dei Dati (*Data Quality*):** funge da barriera protettiva contro errori di digitazione dell'utente o anomalie dei programmi applicativi.
- **Guida nella Progettazione:** aiuta a strutturare gli schemi e a identificare precocemente ridondanze o difetti di normalizzazione.
- **Ottimizzazione delle Query:** l'ottimizzatore interno del DBMS sfrutta la certezza dell'integrità dei dati per semplificare i piani di esecuzione delle interrogazioni, scartando scansioni inutili di blocchi su disco.

Supporto dei Vincoli nei DBMS. I DBMS non offrono il medesimo livello di supporto per qualsiasi tipo di vincolo:

- **Vincoli Dichiarativi Nativi:** sono supportati direttamente dal linguaggio DDL del DBMS (es. chiavi primarie, vincoli di unicità, integrità referenziale e controlli generici con la clausola CHECK). Il DBMS ne verifica automaticamente il rispetto durante ogni operazione di scrittura, respingendo le transazioni illegali.
- **Vincoli Procedurali Esterni:** vincoli complessi che coinvolgono predicati articolati o computazioni aggregate su molte tabelle spesso non sono esprimibili tramite DDL standard. Devono pertanto essere implementati dal programmatore tramite **trigger**, **stored procedure** o controlli inseriti nel software applicativo esterno.

Tassonomia dei Vincoli di Integrità. I vincoli si dividono in due grandi classi:

1. **Vincoli Intra-relazionali:** il predicato deve essere valutato esaminando i dati contenuti all'interno di una singola relazione.
 - *Vincoli su valori (o di dominio):* coinvolgono un singolo attributo isolato.
 - *Vincoli di tupla:* coinvolgono più attributi ma all'interno della medesima tupla.
2. **Vincoli Inter-relazionali:** il predicato coinvolge tuple appartenenti a relazioni distinte (o alla stessa relazione esaminata su istanze diverse), coordinando la coerenza complessiva della base di dati.

2.7 Vincoli Intra-relazionali: Dominio e Tupla

Vincoli di Dominio. Un **vincolo di dominio** è una condizione imposta sul singolo attributo di una relazione, limitando l'insieme dei valori ammissibili a un sottoinsieme proprio del dominio di tipo:

$$t[A] \in D' \subseteq \text{dom}(A)$$

Esempi: imporre che il voto di un esame sia compreso nell'intervallo $[18, 30]$, che l'età di un dipendente sia strettamente positiva o che il campo **Genere** ammetta unicamente i caratteri 'M', 'F' o 'N'.

Vincoli di Tupla. Un **vincolo di tupla** esprime una regola che coinvolge una combinazione di più attributi appartenenti alla medesima riga, **in modo del tutto indipendente** dalle altre tuple memorizzate nella tabella.

Sintassi dei Vincoli di Tupla. Formalmente, un vincolo di tupla viene espresso come un'espressione booleana composta da atomi collegati da connettivi logici (AND, OR, NOT). Ciascun atomo può essere:

- Un confronto tra valori o attributi del dominio (es. $(\text{Voto} \geq 18) \wedge (\text{Voto} \leq 30)$).
- Un confronto tra espressioni aritmetiche calcolate sugli attributi della tupla.

Esempio: Vincolo di Tupla Contabile sulla Busta Paga

Si consideri la tabella **STIPENDI**, dove ogni riga rappresenta la retribuzione mensile di un dipendente suddivisa in imponibile lordo, trattenute fiscali e compenso netto percepito:

Dipendente	StipendioLordo	Ritenute	StipendioNetto
Rossi	55.000	12.500	42.500
Neri	45.000	10.000	35.000
Bruni	47.000	11.000	36.000

Il dominio contabile impone un rigoroso **vincolo di tupla aritmetico**:

$$\text{StipendioLordo} = \text{Ritenute} + \text{StipendioNetto}$$

Tutte le tuple dell'istanza sopra riportata soddisfano il vincolo:

- Tupla di Rossi: $55.000 = 12.500 + 42.500$ (*Verificato*).
- Tupla di Neri: $45.000 = 10.000 + 35.000$ (*Verificato*).
- Tupla di Bruni: $47.000 = 11.000 + 36.000$ (*Verificato*).

Esempio di Violazione del Vincolo. Si supponga che un aggiornamento errato alteri la riga del dipendente Bruni come segue:

Dipendente	StipendioLordo	Ritenute	StipendioNetto
Bruni	50.000	11.000	36.000

Poiché $11.000 + 36.000 = 47.000 \neq 50.000$, la tupla viola il vincolo di integrità di tupla; il DBMS deve respingere la modifica lasciando inalterato lo stato della base di dati.

2.8 Identificazione delle Tuple: Superchiavi e Chiavi

Il Problema dell'Identificazione Univoca. Nel paradigma relazionale basato su valori, non esistono puntatori fisici o numeri di riga impliciti. L'unico modo per individuare una specifica tupla, accedere alle sue informazioni e collegarla ad altre relazioni consiste nel fare leva sui valori dei suoi attributi.

Definizioni Formali: Superchiave e Chiave

Sia r un'istanza di relazione conforme allo schema $R(X)$ con $X = \{A_1, A_2, \dots, A_n\}$:

1. **Superchiave:** un insieme di attributi $K \subseteq X$ è una **superchiave** per r se l'istanza non contiene due tuple distinte $t_1, t_2 \in r$ che coincidano su tutti gli attributi di K :

$$\forall t_1, t_2 \in r, \quad (t_1 \neq t_2) \implies t_1[K] \neq t_2[K]$$

2. **Chiave (o Chiave Candidata):** un insieme di attributi $K \subseteq X$ è una **chiave** per r se è una **superchiave minimale**, ossia:

- K è una superchiave per r .
- Nessun sottoinsieme proprio $K' \subset K$ è una superchiave per r (eliminando anche un solo attributo da K , la proprietà di unicità decade).

Esempio Dettagliato: Ricerca delle Chiavi. Si consideri la seguente istanza della relazione **STUDENTI**:

Matricola	Cognome	Nome	CorsoDiStudi	DataNascita
27655	Rossi	Mario	Ingegneria Informatica	1978/12/05
78763	Rossi	Mario	Ingegneria Informatica	1976/11/03
65432	Neri	Piero	Ingegneria Meccanica	1979/07/10
87654	Neri	Mario	Ingegneria Informatica	1976/11/03
67653	Rossi	Piero	Ingegneria Meccanica	1978/12/05

Analizziamo le possibili combinazioni di attributi:

- $K_1 = \{\text{Matricola}\}$: ciascuna tupla possiede un numero di matricola univoco. È una superchiave ed è composta da un singolo attributo, quindi è **minimale per definizione**: K_1 è una **chiave**.
- $K_2 = \{\text{Cognome}, \text{Nome}, \text{DataNascita}\}$: non esistono due studenti con stesso cognome, stesso nome e medesima data di nascita. È una superchiave. Inoltre:
 - $\{\text{Cognome}, \text{Nome}\}$ non basta (ci sono due *Mario Rossi*).
 - $\{\text{Cognome}, \text{DataNascita}\}$ non basta (ci sono due *Rossi* nati il 1978/12/05).
 - $\{\text{Nome}, \text{DataNascita}\}$ non basta (ci sono due *Mario* nati il 1976/11/03).

Poiché nessun sottoinsieme proprio identifica le tuple, K_2 è minimale ed è una **seconda chiave candidata**.

- $K_3 = \{\text{Matricola}, \text{Cognome}\}$: identifica univocamente le tuple, ma contiene la chiave $\{\text{Matricola}\}$ come sottoinsieme proprio. È una **superchiave non minimale**, quindi non è una chiave.

Schemi, Vincoli e Stati: Il Rischio delle Chiavi Accidentali. Si consideri la combinazione $K_4 = \{\text{Cognome}, \text{CorsoDiStudi}\}$. Nell'istanza specifica mostrata nella tabella precedente, per pura coincidenza, non figurano due tuple uguali su questa coppia di attributi. Possiamo affermare che $\{\text{Cognome}, \text{CorsoDiStudi}\}$ sia una chiave?

Vincoli a Livello di Schema vs Proprietà dello Stato

La risposta è tassativamente **no**:

- I vincoli di integrità riflettono **proprietà del mondo reale modellato** e sono definiti a livello di **schema**.
- Un vincolo deve essere soddisfatto da **tutte le istanze legali ammissibili**, presenti e future, non soltanto dai dati momentaneamente inseriti.
- L'unicità di $\{\text{Cognome}, \text{CorsoDiStudi}\}$ è una coincidenza dello stato attuale (*by chance*): nel mondo reale nulla impedisce a due fratelli o omonimi con il cognome *Rossi* di iscriversi entrambi a *Ingegneria Informatica*.

$\Rightarrow$ Le chiavi non si desumono mai guardando i dati di una tabella campione, ma unicamente dall'analisi dei requisiti e della semantica del dominio.

Il Teorema di Esistenza della Chiave. Poiché per definizione assiomatica del modello relazionale una relazione è un **insieme di tuple distinte**, ne consegue una proprietà teorica di fondamentale importanza:

1. L'insieme formato da **tutti gli attributi della relazione** $X = \{A_1, A_2, \dots, A_n\}$ garantisce l'assenza di tuple identiche ed è perciò sempre una **superchiave**.
2. Poiché l'insieme degli attributi è finito, esso deve contenere almeno un sottoinsieme minimale (che nel caso limite peggiore coincide con l'intero insieme X).

$\Rightarrow$ **Ogni relazione possiede sempre almeno una chiave.**

L'Importanza Sistemistica delle Chiavi. L'esistenza garantita di almeno una chiave assicura due requisiti cardine:

- **Accessibilità univoca:** qualunque elemento informativo all'interno della base di dati può essere reperito in modo deterministico specificando il nome della relazione, il nome dell'attributo e il valore della chiave.
- **Fondamento delle relazioni per valore:** le chiavi costituiscono l'ancora logica per connettere relazioni distinte senza ambiguità.

2.9 Chiavi, Valori NULL e la Chiave Primaria

L'Incompatibilità tra Valori NULL e Chiavi. Se gli attributi appartenenti a una chiave contenessero valori NULL, la funzione stessa della chiave verrebbe irrimediabilmente compromessa:

- **Perdita di identificazione:** due tuple con un valore NULL sulla chiave non possono essere confrontate (NULL non è uguale a NULL nella logica a tre valori), rendendo impossibile verificare se rappresentino la stessa entità o entità distinte.
- **Impossibilità di referenziazione:** un'altra tabella che desideri puntare a quella specifica riga non avrebbe un valore concreto da registrare nella propria colonna di collegamento.

Esempio: La Corruzione dell'Identità delle Tuple con Valori NULL

Si consideri una tabella di studenti con valori NULL distribuiti sulle possibili chiavi:

Matricola	Cognome	Nome	CorsoDiStudi	DataNascita
NULL	NULL	Mario	Ing. Informatica	1978/12/05
78763	Rossi	Mario	Ingegneria Civile	1976/11/03
65432	Neri	Piero	Ingegneria Meccanica	1979/07/10
87654	Neri	Mario	Ing. Informatica	NULL
NULL	Neri	Mario	Ing. Informatica	NULL

Nell'ultima riga, la matricola è ignota e la data di nascita è assente: né {Matricola} né {Cognome, Nome, DataNascita} possono essere utilizzate per distinguere questa persona dagli altri iscritti o per verbalizzare un esame sostenuto.

La Scelta della Chiave Primaria (*Primary Key*). Tra tutte le chiavi candidate individuate su una relazione, il progettista deve selezionarne una specifica che assuma il ruolo di **chiave primaria**:

- La chiave primaria funge da identificatore formale, privilegiato e immutabile delle tuple della tabella.
- Tutte le altre chiavi candidate non prescelte prendono il nome di **chiavi alternative** o secondarie (garantite dal vincolo **UNIQUE** in SQL).
- Si prediligono chiavi compatte (composte da pochi attributi), stabili nel tempo (i cui valori non cambiano mai) e preferibilmente numeriche.

Vincolo di Integrità dell'Entità (*Entity Integrity Constraint*)

Regola Aurea: Gli attributi che compongono la **chiave primaria** di una relazione **non possono mai assumere il valore NULL** (*NOT NULL implicito*).

Convenzione Grafica e Notazionale:

Negli schemi testuali e nelle tabelle relazionali, la chiave primaria viene evidenziata applicando una **sottolineatura** agli attributi che la compongono:

STUDENTI(Matricola, Cognome, Nome, CorsoDiStudi, DataNascita)

2.10 Integrità Referenziale e Chiavi Esterne (*Foreign Key*)

Il Collegamento Inter-relazionale per Valore. Nel modello relazionale, informazioni memorizzate in tabelle distinte vengono poste in correlazione mediante la sovrapposizione di valori condivisi. In particolare, una tabella referenziante include tra i propri attributi i valori della chiave primaria della tabella referenziata.

Affinché la base di dati resti affidabile, tali collegamenti devono essere **coerenti**: non deve mai essere possibile fare riferimento a un'entità inesistente o cancellata.

Definizione Formale di Vincolo di Integrità Referenziale

Siano date due relazioni R_1 e R_2 con rispettivi schemi. Un **vincolo di integrità referenziale** (o **chiave esterna** / *foreign key*) tra un insieme di attributi X di R_1 (detta **relazione referenziante**) e la chiave primaria K di R_2 (detta **relazione referenziata**) impone che:

- Per ogni tupla $t_1 \in r_1$, se il valore di $t_1[X]$ è interamente valorizzato (non contiene NULL), allora deve esistere una tupla $t_2 \in r_2$ tale che:

$$t_1[X] = t_2[K]$$

- In notazione insiemistica di algebra relazionale:

$$\pi_X(r_1) \setminus \{\text{NULL}\} \subseteq \pi_K(r_2)$$

Esempio Completo: Il Sistema di Verbalizzazione delle Infrazioni. Si consideri il sistema di gestione del comando di polizia locale, modellato su tre relazioni:

Relazione INFRAZIONI

<u>Codice</u>	<u>Data</u>	<u>Vigile</u>	<u>Stato</u>	<u>Targa</u>
34321	1995/02/01	3987	Italia	MI39548K
53524	1995/03/04	3295	Italia	AJ046EZ
64521	1996/04/05	3295	Francia	ET234RY
73321	1998/02/05	9345	Finlandia	MMG-418

Relazione VIGILI

<u>Matricola</u>	<u>Cognome</u>	<u>Nome</u>
3987	Rossi	Luca
3295	Neri	Piero
9345	Neri	Mario
7543	Mori	Gino

Relazione AUTO

<u>Stato</u>	<u>Targa</u>	<u>Cognome</u>	<u>Nome</u>
Italia	MI39548K	Rossi	Mario
Italia	AJ046EZ	Verdi	Giuseppe
Francia	ET234RY	Debussy	Claude
Finlandia	MMG-418	Sibelius	Jean

Sulla tabella **INFRAZIONI** insistono due distinti vincoli di integrità referenziale:

1. L'attributo singolo **Vigile** riferenzia la chiave primaria della tabella **VIGILI**:

$$\text{INFRAZIONI}[\text{Vigile}] \xrightarrow{\text{FK}} \text{VIGILI}[\text{Matricola}]$$

2. La coppia di attributi {Stato, Targa} riferenzia la chiave primaria composta della tabella **AUTO**:

$$\text{INFRAZIONI}[\text{Stato}, \text{Targa}] \xrightarrow{\text{FK}} \text{AUTO}[\text{Stato}, \text{Targa}]$$

Violazione dell'Integrità Referenziale. Una violazione del vincolo si verificherebbe qualora venisse inserita nella tabella **INFRAZIONI** una tupla con:

- **Vigile** = 8888, ma tale matricola non compare in **VIGILI**.
- **Stato** = 'Italia', **Targa** = 'XX000YY', ma tale veicolo non risulta registrato nella tabella **AUTO**.

In entrambi i casi, l'infrazione punterebbe al vuoto (*dangling reference*), compromettendo la qualità e l'affidabilità della base di dati.

2.11 Politiche di Reazione alle Violazioni e Vincoli Multipli

Integrità Referenziale e Valori NULL. Il vincolo di chiave esterna non impone necessariamente che il riferimento esista: ammette che l'attributo assuma il valore NULL, a patto che la colonna non sia marcata con il vincolo aggiuntivo NOT NULL.

Esempio: Partecipazione Opzionale a Progetti

Si consideri una base di dati aziendale composta dalle relazioni:

DIPENDENTI(Matricola, Cognome, Progetto)

PROGETTI(Codice, DataInizio, DurataMesi, CostoMilioni)

<u>Matricola</u>	Cognome	Progetto	<u>Codice</u>	DataInizio	Mesi	Costo
34321	Rossi	IDEA	IDEA	2000/01	36	200
53524	Neri	XYZ	XYZ	2001/07	24	120
64521	Verdi	NULL	BOH	2001/09	24	150
73032	Bianchi	IDEA				

La presenza di **NULL** sul dipendente *Verdi* è lecita: significa semanticamente che il dipendente è momentaneamente senza allocazione progettuale. Per i dipendenti che hanno un valore specificato, invece, il codice deve esistere tassativamente nella tabella **PROGETTI**.

Gestione degli Effetti Collaterali: Azioni Compensative. Quando un utente o un programma richiede la **cancellazione** o la **modifica della chiave** di una tupla nella tabella referenziata (es. eliminare il progetto XYZ da **PROGETTI**), le tuple della tabella referenziante (**DIPENDENTI**) rimarrebbero con una chiave esterna orfana, violando il vincolo.

I DBMS commerciali permettono di gestire l'anomalia configurando quattro distinte **azioni compensative**:

1. **Rifiuto / Blocco (*Restrict / No Action*)**: il DBMS impedisce tassativamente l'operazione di cancellazione o aggiornamento, segnalando un errore applicativo fintanto che esistono record figli collegati. È il comportamento predefinito di sicurezza.

2. **Cancellazione a Cascata (*Cascade*)**: l'eliminazione della tupla genitore provoca l'**eliminazione a catena** automatica di tutte le tuple figlie collegate nella tabella referenziante. Se si cancella il progetto XYZ, vengono automaticamente eliminati anche tutti i dipendenti assegnati a quel progetto.
3. **Assegnazione a NULL (*Set NULL*)**: la tupla genitore viene regolarmente eliminata; nei record referenzianti collegati, il valore della chiave esterna viene commutato automaticamente al marcatore speciale NULL (il dipendente *Neri* non viene licenziato, ma la sua colonna *Progetto* diventa NULL).
4. **Assegnazione al Default (*Set Default*)**: il campo della chiave esterna nelle tuple figlie viene reimpostato su un valore predefinito valido specificato nello schema (es. un progetto jolly aziendale generico).

Vincoli Referenziali su Attributi Multipli e Corrispondenza Posizionale. Una relazione può possedere più chiavi esterne indipendenti rivolte verso la medesima tabella di destinazione. Si consideri la relazione **INCIDENTI** che descrive sinistri stradali tra due autovetture:

Relazione INCIDENTI				
<u>Codice</u>	<u>StatoA</u>	<u>TargaA</u>	<u>StatoB</u>	<u>TargaB</u>
34321	Italia	AJ046EZ	Italia	MI39548K
53524	Finlandia	MMG-418	Francia	ET234RY

In questa tabella sussistono due vincoli distinti verso la tabella $AUTO(\underline{Stato}, \underline{Targa})$:

$$\begin{aligned} \mathcal{C}_1 : (\text{StatoA}, \text{TargaA}) &\xrightarrow{\text{FK}} AUTO(\text{Stato}, \text{Targa}) \\ \mathcal{C}_2 : (\text{StatoB}, \text{TargaB}) &\xrightarrow{\text{FK}} AUTO(\text{Stato}, \text{Targa}) \end{aligned}$$

La Rigidità dell'Ordinamento Posizionale nella Definizione dei Vincoli

Quando un vincolo referenziale è definito su una chiave composta formata da più attributi $(A_1, A_2, \dots, A_m) \rightarrow (K_1, K_2, \dots, K_m)$:

- L'**ordine posizionale** degli attributi specificato nella clausola è determinante: l'attributo in posizione i della chiave esterna viene confrontato esclusivamente con l'attributo in posizione i della chiave primaria di destinazione.
- Scambiare l'ordine (es. definire erroneamente $(\text{TargaA}, \text{StatoA}) \rightarrow (\text{Stato}, \text{Targa})$) genera un errore immediato di tipo di dato incompatibile oppure tenta di abbinare una targa a uno stato geografico, corrompendo l'integrità logica del modello.